

WMS-GATE-QC — Starter Project

Project Overview

Program นี้คือระบบ:

- Gate In
- Gate Out
- Weight
- QC
- Offline Mode
- Sync to Dashboard

Technology:

- Electron
- React
- SQLite
- Node.js

Folder Structure

```
WMS-GATE-QC/
|
├── electron/
│   ├── main.js
│   ├── preload.js
│   └── ipc/
│       └── sqlite.js
|
├── react/
│   ├── src/
│   │   ├── pages/
│   │   │   ├── Dashboard.jsx
│   │   │   ├── GateIn.jsx
│   │   │   ├── GateOut.jsx
│   │   │   ├── Weight.jsx
│   │   │   ├── QC.jsx
│   │   │   └── Sync.jsx
│   │   ├── components/
│   │   ├── services/
│   │   ├── utils/
│   │   ├── App.jsx
│   │   └── main.jsx
|
```

```
├─ sqlite/
│  └─ warehouse.db
│    └─ schema.sql
├─ uploads/
│  └─ images/
│    └─ documents/
├─ sync/
│  └─ syncService.js
│    └─ queue.json
├─ package.json
└─ README.md
```

Step 1 — Install Project

Create Project

```
mkdir WMS-GATE-QC
cd WMS-GATE-QC
npm init -y
```

Install Main Packages

```
npm install electron react react-dom
npm install vite
npm install axios express
npm install sqlite3
npm install uuid
npm install react-router-dom
```

Step 2 — package.json

```
{
  "name": "wms-gate-qc",
  "version": "1.0.0",
  "main": "electron/main.js",
  "scripts": {
    "dev": "vite",
```

```
"electron": "electron .",
"start": "concurrently \"npm run dev\" \"npm run electron\""
}
}
```

Step 3 — Electron Main

electron/main.js

```
const { app, BrowserWindow } = require('electron')
const path = require('path')

let mainWindow

function createWindow() {
  mainWindow = new BrowserWindow({
    width: 1400,
    height: 900,
    webPreferences: {
      preload: path.join(__dirname, 'preload.js'),
      contextIsolation: true,
      nodeIntegration: false
    }
  })

  mainWindow.loadURL('http://localhost:5173')
}

app.whenReady().then(() => {
  createWindow()
})
```

Step 4 — Preload

electron/preload.js

```
const { contextBridge } = require('electron')

contextBridge.exposeInMainWorld('electronAPI', {
  appName: 'WMS-GATE-QC'
})
```

Step 5 — SQLite Schema

sqlite/schema.sql

```
CREATE TABLE IF NOT EXISTS gate_logs (  
  id TEXT PRIMARY KEY,  
  plate_no TEXT,  
  driver_name TEXT,  
  gate_in DATETIME,  
  gate_out DATETIME,  
  weight_in REAL,  
  weight_out REAL,  
  status TEXT,  
  sync_status TEXT  
);  
  
CREATE TABLE IF NOT EXISTS qc_logs (  
  id TEXT PRIMARY KEY,  
  gate_id TEXT,  
  product_name TEXT,  
  qty REAL,  
  price REAL,  
  image_path TEXT,  
  document_path TEXT,  
  created_at DATETIME,  
  sync_status TEXT  
);
```

Step 6 — React Main

react/src/main.jsx

```
import React from 'react'  
import ReactDOM from 'react-dom/client'  
import App from './App'  
  
ReactDOM.createRoot(document.getElementById('root')).render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
)
```

Step 7 — App Router

react/src/App.jsx

```
import { BrowserRouter, Routes, Route } from 'react-router-dom'

import Dashboard from './pages/Dashboard'
import GateIn from './pages/GateIn'
import GateOut from './pages/GateOut'
import Weight from './pages/Weight'
import QC from './pages/QC'
import Sync from './pages/Sync'

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Dashboard />} />
        <Route path="/gate-in" element={<GateIn />} />
        <Route path="/gate-out" element={<GateOut />} />
        <Route path="/weight" element={<Weight />} />
        <Route path="/qc" element={<QC />} />
        <Route path="/sync" element={<Sync />} />
      </Routes>
    </BrowserRouter>
  )
}

export default App
```

Step 8 — Dashboard Page

react/src/pages/Dashboard.jsx

```
function Dashboard() {
  return (
    <div>
      <h1>WMS-GATE-QC</h1>

      <ul>
        <li>Gate In</li>
        <li>Gate Out</li>
        <li>Weight</li>
        <li>QC</li>
        <li>Sync</li>
      </ul>
    </div>
  )
}
```

```

    </ul>
  </div>
)
}

export default Dashboard

```

Step 9 — Gate In Page

react/src/pages/GateIn.jsx

```

import { useState } from 'react'
import { v4 as uuidv4 } from 'uuid'

function GateIn() {
  const [plateNo, setPlateNo] = useState('')
  const [driverName, setDriverName] = useState('')

  const saveData = async () => {
    const data = {
      id: uuidv4(),
      plate_no: plateNo,
      driver_name: driverName,
      gate_in: new Date(),
      status: 'IN',
      sync_status: 'PENDING'
    }

    console.log(data)
  }

  return (
    <div>
      <h1>Gate In</h1>

      <input
        placeholder='Plate Number'
        value={plateNo}
        onChange={(e) => setPlateNo(e.target.value)}
      />

      <input
        placeholder='Driver Name'
        value={driverName}
        onChange={(e) => setDriverName(e.target.value)}
      />
    </div>
  )
}

```

```

        <button onClick={saveData}>Save</button>
      </div>
    )
  }

  export default GateIn

```

Step 10 — Sync Page

react/src/pages/Sync.jsx

```

import { useState } from 'react'
import axios from 'axios'

function Sync() {
  const [ip, setIp] = useState('192.168.1.10')

  const syncData = async () => {
    try {
      await axios.post(`http://${ip}:3000/api/sync/gate`, {
        message: 'sync test'
      })

      alert('Sync Success')
    } catch (error) {
      alert('Sync Failed')
    }
  }

  return (
    <div>
      <h1>Sync Dashboard</h1>

      <input
        value={ip}
        onChange={(e) => setIp(e.target.value)}
      />

      <button onClick={syncData}>
        Send Data
      </button>
    </div>
  )
}

export default Sync

```

Step 11 — Sync Queue

sync/queue.json

```
[ ]
```

Step 12 — Sync Service

sync/syncService.js

```
const axios = require('axios')

async function sendData(ip, payload) {
  try {
    const response = await axios.post(
      `http://${ip}:3000/api/sync/gate`,
      payload
    )

    return response.data
  } catch (error) {
    console.log(error)
  }
}

module.exports = {
  sendData
}
```

Step 13 — Features To Build Next

Gate

- Auto Running Number
 - Camera
 - Print Slip
 - Weight Scale
-

QC

- Upload Image
 - Upload PDF
 - Multi Product
-

Sync

- Auto Retry
 - Background Sync
 - Queue System
-

Step 14 — Dashboard API That This Program Will Connect To

```
POST /api/sync/gate
POST /api/sync/qc
```

Program Status

Completed Starter

- Electron Structure
 - React Structure
 - SQLite Schema
 - Sync System Base
 - Gate In Page
 - Dashboard Page
 - Sync Page
-

Next Step

หลังจาก Program 1 เสร็จ:

1. ทำระบบ SQLite Save จริง
2. ทำระบบ Upload File
3. ทำระบบ Weight
4. ทำระบบ Print
5. ทำระบบ Dashboard Server
6. ทำ Program 2 Stock Office

FULL FILES TO CREATE IN VISUAL STUDIO CODE

Create Folder

```
mkdir WMS-GATE-QC
cd WMS-GATE-QC
```

Root Files

package.json

```
{
  "name": "wms-gate-qc",
  "version": "1.0.0",
  "main": "electron/main.js",
  "scripts": {
    "dev": "vite",
    "electron": "electron .",
    "start": "concurrently \"npm run dev\" \"npm run electron\""
  },
  "dependencies": {
    "axios": "^1.6.0",
    "electron": "^30.0.0",
    "express": "^4.19.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.22.0",
    "sqlite3": "^5.1.7",
    "uuid": "^9.0.1",
    "vite": "^5.2.0"
  },
  "devDependencies": {
    "concurrently": "^8.2.2"
  }
}
```

Create Electron Folder

```
/electron
```

electron/main.js

```
const { app, BrowserWindow } = require('electron')
const path = require('path')

function createWindow() {
  const win = new BrowserWindow({
    width: 1400,
    height: 900,
    webPreferences: {
      preload: path.join(__dirname, 'preload.js')
    }
  })

  win.loadURL('http://localhost:5173')
}

app.whenReady().then(() => {
  createWindow()
})
```

electron/preload.js

```
const { contextBridge } = require('electron')

contextBridge.exposeInMainWorld('electronAPI', {
  appName: 'WMS-GATE-QC'
})
```

Create React Folder

```
/react/src
```

react/index.html

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8" />
  <title>WMS-GATE-QC</title>
```

```
</head>
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

react/src/main.jsx

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
)
```

react/src/App.jsx

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom'

import Dashboard from './pages/Dashboard'
import GateIn from './pages/GateIn'
import Sync from './pages/Sync'

function App() {
  return (
    <BrowserRouter>
      <div>
        <nav>
          <Link to="/">Dashboard</Link> |
          <Link to="/gate-in">Gate In</Link> |
          <Link to="/sync">Sync</Link>
        </nav>

        <Routes>
          <Route path="/" element={<Dashboard />} />
          <Route path="/gate-in" element={<GateIn />} />
          <Route path="/sync" element={<Sync />} />
        </Routes>
      </div>
    </BrowserRouter>
  )
}
```

```
}  
  
export default App
```

Create Pages Folder

```
/react/src/pages
```

react/src/pages/Dashboard.jsx

```
function Dashboard() {  
  return (  
    <div>  
      <h1>WMS-GATE-QC Dashboard</h1>  
  
      <ul>  
        <li>Gate In</li>  
        <li>Gate Out</li>  
        <li>Weight</li>  
        <li>QC</li>  
        <li>Sync</li>  
      </ul>  
    </div>  
  )  
}  
  
export default Dashboard
```

react/src/pages/GateIn.jsx

```
import { useState } from 'react'  
import { v4 as uuidv4 } from 'uuid'  
  
function GateIn() {  
  const [plateNo, setPlateNo] = useState('')  
  const [driverName, setDriverName] = useState('')  
  
  const saveData = () => {  
    const data = {  
      id: uuidv4(),  
      plate_no: plateNo,  
      driver_name: driverName,  
      weight: 0,  
      qc: 0,  
      sync: 0,  
    }  
    // Save data to database  
  }  
}
```

```

        driver_name: driverName,
        gate_in: new Date(),
        status: 'IN'
    }

    console.log(data)

    alert('Saved')
}

return (
    <div>
        <h1>Gate In</h1>

        <input
            placeholder='Plate Number'
            value={plateNo}
            onChange={(e) => setPlateNo(e.target.value)}
        />

        <br /><br />

        <input
            placeholder='Driver Name'
            value={driverName}
            onChange={(e) => setDriverName(e.target.value)}
        />

        <br /><br />

        <button onClick={saveData}>Save</button>
    </div>
)
}

export default GateIn

```

react/src/pages/Sync.jsx

```

import { useState } from 'react'
import axios from 'axios'

function Sync() {
    const [ip, setIp] = useState('192.168.1.10')

    const sendData = async () => {
        try {
            await axios.post(`http://${ip}:3000/api/sync/gate`, {

```

```

        message: 'Hello Dashboard'
      })

      alert('Sync Success')
    } catch (error) {
      alert('Sync Failed')
    }
  }
}

return (
  <div>
    <h1>Sync Dashboard</h1>

    <input
      value={ip}
      onChange={(e) => setIp(e.target.value)}
    />

    <br /><br />

    <button onClick={sendData}>
      Send Data
    </button>
  </div>
)
}

export default Sync

```

Create SQLite Folder

```
/sqlite
```

sqlite/schema.sql

```

CREATE TABLE IF NOT EXISTS gate_logs (
  id TEXT PRIMARY KEY,
  plate_no TEXT,
  driver_name TEXT,
  gate_in DATETIME,
  gate_out DATETIME,
  status TEXT
);

```

Create Vite Config

vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  root: './react'
})
```

Install Additional Package

```
npm install @vitejs/plugin-react
```

RUN PROJECT

Install Dependencies

```
npm install
```

Start React

```
npm run dev
```

Open Electron

Open new terminal:

```
npm run electron
```


FINAL RESULT

เมื่อรันสำเร็จ:

- Electron WinApp จะเปิด
- Dashboard จะขึ้น
- เข้าหน้า Gate In ได้
- เข้า Sync ได้
- พร้อมเริ่มพัฒนาระบบจริงต่อ